

Hub de Negocios Multi-Tenant

Base de microservicios en .NET 8 con EF Core y PostgreSQL

Este documento explica qué se construyó, por qué se tomó cada decisión y —sobre todo— **qué no está verificado todavía**. Está escrito para repartirse entre las tres personas que van a trabajar sobre esta base.

5 / 10

SERVICIOS COMPLETOS

142

TESTS EN VERDE

0

AVISOS Y ERRORES

Qué leer según tu rol

Backend

Secciones 2, 3 y 4. Cómo está partido el código, por qué hay un núcleo compartido, cómo se emiten eventos sin perderlos y cómo se llama a otro servicio sin arrastrarlo en la caída.

Base de datos

Secciones 5 y 6. Una base por servicio, tipos elegidos, índices, migraciones y el bloqueo pesimista que evita vender dos veces lo mismo.

Ciberseguridad

Secciones 7 y 8, y muy especialmente la 9. Ahí está el aislamiento entre clientes, el tratamiento de credenciales y la lista de lo que aún *no* se ha podido probar.

ANTES DE FIARSE DE NADA DE ESTO

Nada se ha ejecutado nunca contra un PostgreSQL real. El código compila y los tests de lógica pasan, pero **las migraciones no se han aplicado y las políticas de aislamiento no se han probado**. La sección 9 lo detalla. Es lo primero que hay que cerrar.

1. Qué hay construido, y qué no

El objetivo era levantar diez microservicios siguiendo la especificación de `prompts-efcore-microservicios.md` y la arquitectura de `ExplicacionTec.md`. Están terminados cinco.

Servicio	Estado	Tests	Qué hace
<code>tenant-service</code>	Completo	22	Alta, suspensión y resolución de negocios por subdominio
<code>identity-service</code>	Completo	16	Registro, login, tokens, roles y permisos
<code>catalog-service</code>	Completo	57	Productos, precios con histórico y búsqueda
<code>order-service</code>	Completo	22	Órdenes validadas contra el catálogo
<code>reservations-service</code>	Completo	25	Franjas horarias y reservas sin sobreventa
<code>billing-service</code>	Pendiente	—	Facturación y pasarelas de pago
<code>inventory-service</code>	Pendiente	—	Stock con reserva y confirmación
<code>notification-service</code>	Pendiente	—	Correo y SMS por plantillas
<code>checkout-service</code>	Pendiente	—	Carrito y paso a orden
<code>landing-builder-service</code>	Pendiente	—	Páginas públicas por bloques

Los cinco pendientes **no están a medias en la rama principal**, y es a propósito: sin punto de entrada no compilan, y una rama donde `dotnet build` falla no le sirve a nadie. El modelado de dominio que se alcanzó a escribir está aislado en la rama `wip/servicios-pendientes`, y `docs/pendiente.md` tiene el guion de cada uno con las trampas ya identificadas.

Dónde está

Repositorio `Hexor-Labs/ReadMeOffProyect`, rama `feature/efcore-microservicios`, pull request #1. La rama `main` está intacta.

2. Cómo está organizado el código

Monorepo, no diez repositorios. Es lo que recomienda `ExplicacionTec.md` para un equipo de tres personas: un solo sitio donde buscar, un solo CI que configurar bien, y los cambios que cruzan servicios caben en un commit.

```
hub-negocios/
├── packages/SharedKernel/  infraestructura común a los 10 servicios
├── services/               un microservicio por carpeta
├── tests/                 un proyecto de pruebas por servicio
├── docker/               arranque de Postgres (bases y roles)
├── docs/                 arquitectura, diagramas y pendientes
└── docker-compose.yml    entorno local completo
```

Cada servicio sigue arquitectura hexagonal en tres capas:

Capa	Qué contiene	De qué depende
Domain/	Entidades, objetos de valor y puertos (interfaces)	De nada. Ni EF Core ni ASP.NET
Application/	Casos de uso	Solo del dominio
Infrastructure/	EF Core, controladores, clientes HTTP	De todo lo anterior

Que el dominio no dependa de nada tiene una consecuencia muy concreta y muy práctica: **los tests de lógica de negocio corren sin base de datos y sin Docker**, en milisegundos. Por eso los dobles de prueba están escritos a mano en vez de con una librería de simulación: si hiciera falta simular un `DbContext` para probar una regla de negocio, querría decir que la regla se ensució con infraestructura.

Por qué hay un núcleo compartido

La propia especificación pedía comparar los `Program.cs` de varios servicios y desconfiar si divergían. Un paquete compartido convierte esa revisión manual en algo que **no puede fallar**: no divergen porque es literalmente el mismo código. El arranque de un servicio queda así:

```
builder.AddHubDefaults("order-service");    // logs, JWT, límites, health
builder.AddHubPersistence<OrderDbContext>(); // EF, interceptores, outbox

// ... solo lo propio de este servicio ...

app.UseHubDefaults();                      // middleware en el orden correcto
```

REGLA PARA NO ROMPER ESTO

En el núcleo compartido vive **infraestructura transversal, nunca lógica de negocio**. Un núcleo que empieza a saber de órdenes o de reservas es el primer paso para volver a tener un monolito, solo que repartido en diez procesos y con la latencia de red de propina.

3. Cómo se comunican los servicios

Dos caminos, y la elección entre uno y otro no es de estilo:

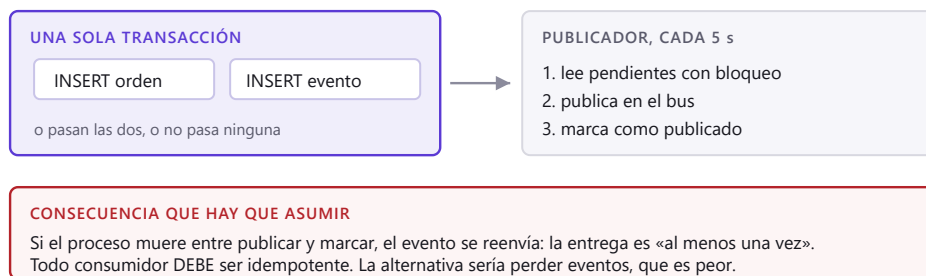
Camino	Cuándo	Ejemplo
Evento asíncrono vía outbox	Cuando algo <i>ya pasó</i> y otros deben enterarse	<code>order.created</code> despierta a reservas, stock y notificaciones
HTTP síncrono con Polly	Cuando se necesita una respuesta <i>antes</i> de continuar	Órdenes pregunta al catálogo el precio real antes de crear la orden

El patrón Outbox, y por qué no es opcional

Un caso de uso tiene que hacer dos cosas: guardar el cambio y avisar al resto del sistema. No hay forma de hacer esas dos cosas atómicamente contra dos sistemas distintos. Si el proceso muere en medio, o se confirmó la venta

y nadie se enteró, o se avisó de una venta que no existe.

La solución es escribir el evento como **una fila más de la misma transacción**:



Flujo del patrón Outbox y su contrapartida

El publicador usa `SELECT ... FOR UPDATE SKIP LOCKED`, que es lo que permite que varias instancias del servicio publiquen a la vez sin pisarse: cada una se lleva las filas que bloquea y las demás pasan de largo en vez de esperar turno.

Hoy el publicador escribe en el log en vez de en un bus real (`LoggingEventBusPublisher`). Es deliberado, para poder desarrollar sin levantar RabbitMQ. Conectarlo de verdad es escribir una implementación de `IEventBusPublisher` y cambiar una línea.

Llamadas síncronas: tres defensas, tres problemas distintos

Cuando órdenes llama al catálogo, esa llamada lleva tres protecciones que suelen confundirse entre sí:

Defensa	Valor	Qué problema resuelve
Tiempo máximo de espera	3 s	Que una petición nuestra no se quede colgada esperando a quien no va a contestar. Sin esto los hilos se acumulan y caemos <i>nosotros</i> por culpa de otro.
Reintentos con espera creciente	3 · 1s/2s/4s	El fallo pasajero: un despliegue en marcha, un pico de red. La espera crece porque reintentar de inmediato contra un servicio saturado es echar gasolina al fuego.
Cortacircuitos	5 fallos / 30 s	El fallo sostenido. Deja de intentarlo y responde al instante. Protege a los dos: al otro, que deja de recibir tráfico mientras se recupera, y a nosotros, que dejamos de gastar 3 s por petición en algo que ya sabemos que falla.

POR QUÉ LAS TRES JUNTAS

Sin el cortacircuitos, **los reintentos empeoran una caída**: multiplican por tres la carga justo cuando el otro servicio menos la aguanta. Es un error muy común y muy caro.

4. Convenciones de código

Regla	Motivo
Tablas en <code>snake_case</code> , mapeadas explícitamente	Convención de Postgres; evita tener que citar identificadores
Enums guardados como texto , no como número	Un <code>2</code> en una columna no dice nada a las tres de la mañana. Y reordenar el enum en C# reinterpretaría en silencio todas las filas ya guardadas
Dinero en <code>decimal</code> / <code>numeric(18,2)</code>	En coma flotante binaria 0,1 no es 0.1: los céntimos dejan de cuadrar
Fechas siempre en UTC	Un sistema multi-tenant acaba sirviendo a varios husos
DTOs en los endpoints, nunca entidades de dominio	Si el endpoint aceptara la entidad, el cliente podría mandar <code>"status": "Paid"</code> en el JSON y el enlazador lo asignaría sin preguntar
Los avisos del compilador son errores	En diez servicios, un aviso que nadie mira se multiplica por diez y deja de verse
Versiones de paquetes centralizadas	Si cada servicio fija las suyas, dos acaban corriendo EF Core distintos y el mismo código se comporta distinto

Formato de error único

Todos los servicios devuelven la misma forma, para que el frontend no aprenda diez maneras de saber que algo falló:

```
{
  "error": {
    "code": "ORDER_ITEM_UNAVAILABLE",
    "message": "Ya no están disponibles: Botella premium",
    "details": {},
    "trace_id": "0af7651916cd43dd8448eb211c80319c"
  }
}
```

El `code` es contrato estable y el `message` es para personas: mezclarlos obliga al frontend a comparar cadenas en español, que es lo que se rompe el día que alguien corrige una tilde. El `trace_id` viaja entre servicios en la cabecera `X-Correlation-Id` y se devuelve **siempre**, también en los errores: es el número que un cliente puede leernos por teléfono.

5. Base de datos

Una base por servicio

Diez bases sobre un mismo servidor físico, para ahorrar coste al principio sin renunciar al aislamiento lógico:

```
tenant_db  identity_db  catalog_db  order_db    billing_db
reservations_db  inventory_db  checkout_db  landing_db  notification_db
```

REGLA QUE NO SE PUEDE ROMPER

Ningún servicio consulta la base de otro. Si órdenes necesita datos del catálogo, los pide por API o los recibe por evento. Un `JOIN` entre bases de dos servicios no se puede deshacer después sin reescribir medio sistema, porque para cuando duele ya hay veinte consultas que dependen de él.

Por eso cada línea de una orden guarda una **foto** del producto —nombre, precio, atributos— y no una referencia. Es duplicación a propósito: si mañana el negocio sube el precio, la factura de ayer tiene que seguir diciendo lo que el cliente pagó ayer. Leer el precio en vivo reescribiría el histórico de ventas, que además de incorrecto es ilegal en la mayoría de sitios.

Dos roles de Postgres, y esto es lo más importante de la sección

Rol	Quién lo usa	Por qué
hub	Migraciones y publicador de outbox	Es dueño de las tablas. Necesita leer eventos de <i>todos</i> los clientes a la vez
hub_app	El servicio, al atender peticiones	No es dueño de nada, así que el aislamiento por fila sí se le aplica

SI ESTO SE CONFIGURA MAL, EL AISLAMIENTO NO EXISTE

En PostgreSQL, **el dueño de una tabla se salta las políticas de Row-Level Security por defecto**. Si el servicio se conectara con el usuario dueño, las políticas estarían escritas pero no se aplicarían nunca: parecería que hay aislamiento y no lo habría, sin que nada lo avisara. Verificar qué usuario hay en la cadena de conexión de producción es la comprobación número uno.

Tipos elegidos y por qué

Uso	Tipo	Motivo
Atributos libres, marca, contenido de bloques	<code>jsonb</code>	Nativo, no texto serializado: se puede consultar dentro del JSON desde SQL sin leer y deserializar la fila entera
Importes	<code>numeric(18,2)</code>	Exacto. Nunca <code>float</code>
Estados	<code>varchar(20)</code>	Legible en consultas manuales
Etiquetas, listas de imágenes	<code>text[]</code> + índice GIN	Array nativo de Postgres; el GIN hace que filtrar por etiqueta escale
Identificadores	<code>uuid</code>	Generables sin ir a la base, imprescindible en sistemas distribuidos

Índices que conviene conocer

- **Parcial sobre la outbox** — solo indexa lo pendiente (`WHERE published = false`). La tabla se llena de eventos ya publicados que nunca se releen: indexarlos todos sería pagar por mantener un índice enorme del que siempre se usa la misma esquina.
- **Únicos por tenant, no globales** — el número de orden, el slug, el correo del usuario. Dos negocios pueden tener cada uno su `ORD-2026-000001` , y el mismo correo puede ser cliente de dos negocios distintos.
- **Único con filtro** — el dominio propio del tenant es único *cuando existe* (`WHERE custom_domain IS NOT NULL`). Sin el filtro, el segundo cliente sin dominio chocaría con el primero.

Migraciones

Una migración inicial por servicio, generada con EF Core y editada a mano para añadir lo que EF no sabe expresar: las políticas de seguridad por fila y las restricciones `CHECK` . Cada servicio gestiona solo las suyas; ninguna migración toca la base de otro.

6. Concurrencia: el problema de vender dos veces lo mismo

Es el punto más delicado del sistema y el que más caro sale si se hace mal. Dos clientes reservan la última mesa en el mismo instante:

Cliente A	Cliente B
lee cupo → queda 1	lee cupo → queda 1
escribe cupo = 0 ✓	escribe cupo = 0 ✓
Resultado: dos reservas confirmadas para una sola mesa.	

Un `if (cupo > 0)` en el código **no lo evita**, por muy bien escrito que esté: entre leer y escribir cabe otra transacción. La solución está en la base de datos, no en la aplicación:

```
SELECT * FROM time_slots
WHERE id = $1 AND tenant_id = $2
FOR UPDATE                -- ← bloquea la fila hasta el COMMIT
```

Con `FOR UPDATE`, B se queda esperando en la lectura hasta que A termina, y cuando por fin lee, ve el cupo ya en cero. Va en SQL a mano porque EF Core no tiene forma de expresar un bloqueo de fila, y los valores van **siempre como parámetros**, nunca concatenados.

DEFENSA EN PROFUNDIDAD

Además del bloqueo hay una restricción `CHECK` que impide que el cupo baje de cero por ningún camino. El bloqueo es la defensa; el `CHECK` es la red por si algún día alguien escribe por otra vía.

El mismo patrón hará falta en `inventory-service` cuando se implemente. Está documentado en `docs/pendiente.md`.

7. Aislamiento entre clientes

Es la propiedad de seguridad más importante de un SaaS multi-tenant: que el negocio A no pueda ver jamás datos del negocio B. No se confía a una sola medida.

1 · Filtro global de EF Core

Toda consulta LINQ lleva el filtro de cliente puesto sin que nadie lo escriba.
Se escapa por: IgnoreQueryFilters(), SQL crudo

2 · Interceptor de escritura

Impide guardar una fila en el cliente equivocado, con un error legible.
Se escapa por: no cubre lecturas

3 · Row-Level Security de PostgreSQL

Filtra dentro del motor. Es lo único que sigue en pie ante SQL a mano.
Se escapa por: si la aplicación se conecta como DUEÑA de la tabla

El identificador del cliente entra al sistema UNA sola vez, en el borde, leído del token ya validado, y de ahí en adelante viaja solo en el contexto de ejecución. Nunca es un parámetro que haya que recordar.

Tres capas, porque cada una deja pasar lo que tapan las otras

Tres fallos sutiles que se evitaron

Los tres están comentados en el código, porque son el tipo de cosa que alguien «simplifica» sin saber lo que está quitando.

El filtro no puede apuntar al contexto de base de datos

El patrón que sale en casi todos los tutoriales guarda el cliente en un campo del contexto y compara contra él. El problema es que EF Core **cachea el modelo por tipo de contexto**: la segunda petición reutiliza un modelo que sigue apuntando al contexto de la primera, y acaba filtrando por el cliente equivocado. Aquí se compara contra un valor estático, sin capturar instancia alguna.

Hay que limpiar el identificador al soltar la conexión

Las conexiones salen de un pool y se reutilizan. Si al no haber cliente no se limpiara el valor, **una petición anónima heredaría el cliente de la anterior** que usó esa misma conexión física. La política está escrita para que, sin valor, no devuelva ninguna fila: el estado por defecto es negar, no mostrar todo.

La outbox queda deliberadamente fuera del filtro

La lee un proceso de fondo que publica eventos de todos los clientes y que, por definición, corre sin cliente en contexto. Con filtro no vería una sola fila y ningún evento saldría nunca. Es una excepción consciente y está documentada donde toca.

8. Credenciales y superficie expuesta

Contraseñas

BCrypt con factor de coste 12. El factor no es decorativo: cada unidad **duplica** el trabajo. Doce deja cada comprobación en unos cientos de milisegundos —imperceptible al entrar, carísimo para quien pruebe millones de contraseñas contra una tabla robada—. Subirlo cuando las máquinas mejoren es lo que mantiene esto cierto dentro de cinco años.

ATAQUE POR TIEMPO, Y CÓMO SE CERRÓ

El login devuelve el mismo error tanto si el correo no existe como si la contraseña es incorrecta. Eso es lo habitual y **no basta**: si cuando el correo no existe la respuesta llega en un milisegundo y cuando sí existe tarda doscientos, cualquiera puede averiguar qué direcciones están dadas de alta cronometrando las respuestas. El mensaje es idéntico, pero el reloj lo cuenta todo.

Cuando el usuario no existe se comprueba igualmente contra un hash de descarte con el mismo factor de coste, para gastar el mismo tiempo. **Hay un test que lo mide** y falla si alguien elimina esa rama por parecerle código muerto.

Tokens

	Token de acceso	Token de renovación
Duración	15 minutos	14 días
Dónde va	Cuerpo de la respuesta	Cookie <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Strict</code>
Qué se guarda	Nada	Solo el hash , nunca el token
¿Revocable?	No, hasta que caduca	Sí. Se rota en cada uso

El token de acceso lleva firmados el usuario, **el cliente** y el rol, y los demás servicios lo validan sin preguntarle a nadie. Eso es lo que permite escalar sin que identidad sea el cuello de botella, y es también la razón de que no se pueda revocar antes de tiempo: quince minutos es el equilibrio entre no molestar al usuario y que un token robado sirva de poco.

Del token de renovación solo se guarda el hash. Si se filtrara la tabla —una copia de seguridad mal guardada, un volcado para depurar— lo robado no serviría para entrar. Y al viajar en cookie `HttpOnly` , JavaScript no puede leerlo: es la diferencia práctica entre que un XSS robe la sesión y que no pueda.

Superficie expuesta sin autenticación

Por defecto **todo endpoint exige autenticación**. Lo público se marca uno por uno, que es un olvido visible; al revés el olvido es invisible y deja una puerta abierta. Hoy son anónimos:

Endpoint	Por qué	Qué lo protege
<code>/health</code> , <code>/ready</code>	El orquestador los consulta sin credenciales	No exponen datos
Resolver negocio por subdominio	El gateway lo llama antes de que exista sesión	Devuelve solo datos ya públicos. Un negocio eliminado responde igual que uno inexistente, para no convertirlo en un listado de quién estuvo y se fue
Registro y login	Quien entra aún no tiene token	Límite de peticiones y bloqueo tras 5 intentos

Otras medidas ya puestas

- **Límite de peticiones** repartido por cliente cuando lo hay y por IP cuando no. Contar solo por IP castigaría a todo un negocio detrás de una misma oficina; contar solo por cliente dejaría sin freno el tráfico anónimo, que

es justo el que conviene frenar.

- **Bloqueo temporal** tras cinco intentos fallidos, quince minutos. Temporal y no permanente a propósito: un bloqueo que solo levanta un administrador convierte un ataque contra una cuenta en una forma fácil de dejar a alguien fuera para siempre.
- **Los errores inesperados no salen al cliente.** Un error de Postgres sin filtrar cuenta nombres de tablas y columnas, y a veces datos. Fuera va un código genérico y el identificador de traza; la traza completa va al log.
- **Sin secretos en el repositorio.** `.env` está ignorado desde el primer commit, y el servicio **no arranca** si falta la clave de firma: preferimos no arrancar a arrancar aceptando cualquier token.
- **Consentimiento con fecha** (Ley 1581 de Habeas Data). Sin fecha, el consentimiento no se puede acreditar.
- **Contenedores con usuario sin privilegios**, nunca root.

9. Lo que NO está verificado

Esta sección es la más importante del documento. Todo lo anterior describe lo que el código *dice* que hace; aquí está lo que todavía nadie ha comprobado que haga.

NADA SE HA EJECUTADO CONTRA UN POSTGRESQL REAL

En la máquina donde se escribió esto no había Docker. Las migraciones se generan y compilan, pero **no se han aplicado nunca a una base de datos**. Es perfectamente posible que alguna falle al ejecutarse.

Qué	Estado	Cómo cerrarlo
Las migraciones se aplican sin error	Sin probar	<code>docker compose up -d</code> y <code>dotnet ef database update</code> por servicio
Las políticas de aislamiento funcionan	Sin probar	Test con dos clientes: escribir datos con uno e intentar leerlos con el otro. Es el primer test que hay que escribir.
El bloqueo evita la doble reserva	Sin probar	Ya hay un test de concurrencia escrito con Testcontainers, marcado para saltarse. Basta con Docker para ejecutarlo
La aplicación se conecta como rol no dueño	Configurado	Está en <code>docker-compose.yml</code> , pero hay que verificarlo en el entorno real
Los eventos llegan a un bus	No implementado	Hoy se escriben en el log. Falta el adaptador de RabbitMQ
Lógica de negocio de los 5 servicios hechos	142 tests	Verificado. No necesita base de datos

Riesgos abiertos, ordenados por gravedad

1. **Que la aplicación acabe conectándose como dueña de las tablas en producción.** Anularía el aislamiento por fila sin que nada lo avisara, y la aplicación funcionaría con normalidad. Es el fallo más grave posible en este diseño y el más silencioso.
2. **Que una política de aislamiento esté mal escrita.** Hasta que no se pruebe contra Postgres con dos clientes, no hay garantía de que filtre.
3. **Que un consumidor de eventos no sea idempotente.** La entrega es «al menos una vez»: descontaría stock dos veces o mandaría dos correos.
4. **Falta rotación de secretos.** La política de 90 días está documentada pero no implementada.

10. Cómo seguir

Antes de escribir una línea más

1. Levantar la infraestructura y aplicar las migraciones. Arreglar lo que falle.

2. **Escribir el test de aislamiento entre clientes.** Dos clientes, datos cruzados, y comprobar que ninguno ve los del otro ni con SQL a mano.
3. Ejecutar el test de concurrencia de reservas, que ya está escrito.
4. Revisar con qué usuario se conecta cada servicio en cada entorno.

Después

5. Implementar los cinco servicios que faltan siguiendo `docs/pendiente.md`, que ya recoge las trampas de cada uno.
6. Escribir el adaptador real del bus de eventos.
7. Montar el CI: compilación y tests obligatorios antes de fusionar.
8. Proteger la rama principal: sin empuje directo, con revisión.

Arranque local

```
cp .env.example .env          # rellenar los valores
docker compose up -d          # Postgres (10 bases) + RabbitMQ

dotnet build HubNegocios.sln   # 0 errores, 0 avisos
dotnet test HubNegocios.sln    # 142 tests
```

Dónde está cada cosa

Documento	Contenido
<code>README.md</code>	Arranque, convenciones y estado
<code>docs/arquitectura.md</code>	Diagramas y decisiones de fondo
<code>docs/pendiente.md</code>	Guion de los cinco servicios que faltan
<code>ExplicacionTec.md</code>	Especificación original del proyecto
README por servicio	Variables de entorno, endpoints, eventos y runbook

Repositorio `Hexor-Labs/ReadMeOffProject` · rama `feature/efcore-microservicios` · pull request #1